Design Rationale REQ 5 Member B: Foraging Behavior

Member B provides food selection, reachable-target search, movement, pickup and eating for SporeBeast. ForagingState integrates these behaviours with the shared CreatureState framework. This design adds purposeful food seeking while reusing existing food effects and engine actions.

Food Selection and Consumption

Design 1:

Put concrete food checks and duplicated healing rules directly in ForagingState. Once the creature reaches a selected item, remove it and apply a fixed health bonus.

Pros	Cons
Simple local flow: Food recognition and its immediate result are visible in one method, with few collaborators.	Coupled rules: Adding food types changes the state. A fixed bonus loses existing effects such as strawberry poisoning and melon maximum-health increases. Duplicating food logic makes the two implementations likely to diverge.

Design 2:

Use EdibleFoodFilter for eligibility, PickUpFoodAction for collection and the existing ConsumeAction for eating. The default diet accepts portable ConsumableProduce. An injected Predicate<Item> can change the diet without rewriting search or movement.

Pros	Cons
SRP and OCP: Eligibility, collection and consumption have separate responsibilities. Reusing ConsumeAction and Consumable preserves each food's own effects and avoids duplicating them (DRY).	Restricted compatibility: Consumable alone does not guarantee that a food supports a health-only creature. Legacy foods that access stamina are excluded. Extending the diet still requires checking each food's consumption contract.

Selected design: Design 2. Each turn first consumes one carried food, otherwise collects one on the current tile, otherwise moves toward food. Pickup and eating take separate turns. PickUpFoodAction rechecks the actor's location, consciousness, item presence and eligibility before transferring ownership, preventing a stale action from duplicating food already taken by another actor.

The default constructor shares one filter with the finder and pickup action. This keeps the meaning of "edible" consistent across those operations. Rotten strawberries remain eligible; their existing poison effect is preserved rather than silently replaced with healing.

Nearest Reachable Food and Movement

Design 1:

Choose food by coordinate distance and move directly toward it, or wander until the creature encounters an edible item.

Pros	Cons
Low search overhead: Coordinate comparisons are inexpensive, and a simple movement rule needs little supporting code.	Poor reachability decisions: The geometrically closest food may be behind an impassable tile or another actor. Wandering does not implement purposeful nearest-food seeking and cannot guarantee progress toward a reachable target.

Design 2:

NearestEdibleFoodFinder uses breadth-first search over enterable exits on the current map. DeterministicPathfinder searches a route to the chosen tile and returns its first step. Neighbours are ordered by x, then y, making choices reproducible when several routes or targets have equal distance.

Pros	Cons
Correct distance model: With one move per exit, BFS finds food using the fewest legal movement steps. Occupied and impassable tiles are excluded. Separating target selection from routing allows the pathfinder to be reused by FoodLureEffect.	Repeated work: Finding a target and finding its route use separate BFS traversals, and neighbour-ordering logic appears in both helpers. Recomputing routes each turn costs more than caching; larger maps may justify a shared search result.

Selected design: Design 2. The route reflects the engine's legal movement rules, including diagonal exits, rather than assuming Manhattan distance. Each search visits a location at most once. Location objects identify visited tiles, and cross-map exits are excluded to prevent matching unrelated tiles with the same coordinates.

The state recomputes its target every turn instead of retaining a stale food reference. If another actor removes the food, the next decision searches the current map again. When no food is reachable, createTurnAction() can return DoNothingAction; the state-transition decision can instead return to camouflage when no carried food remains.

BFS keeps the algorithm understandable for the current unweighted maps (KISS). This choice does not assume that it would remain optimal if future terrain introduced different movement costs.

State Transitions and Entry Effects

Design 1:

Let the ordinary foraging turn method directly change the creature's state and trigger the environmental effect.

Pros	Cons
Fewer lifecycle methods: The decision and its immediate consequences can be followed in one local block.	Timing coupling: Mixed responsibilities can repeat entry effects every turn or cause multiple transitions. Each state must reproduce lifecycle rules.

Design 2:

Implement getNextState(), onEnter() and createTurnAction() through CreatureState. SporeBeast applies one state decision, calls onEnter() only when the state object changes, and then requests the current state's ordinary action.

Pros	Cons
Clear responsibilities: State selection, entry and normal actions stay distinct. The actor uses CreatureState; FoodLureEffect handles relocation separately from movement and consumption.	Framework coupling: Timing depends on SporeBeast. Concrete helpers and the static BaseStates registry remain dependencies, so dependency inversion applies at the state boundary, not throughout the module.

Selected design: Design 2. ThreatScanner inspects nearby players and hostile creatures within square radius two. Two or more threats cause fleeing; one causes territorial behaviour. Without threats, absent carried and reachable food causes camouflage; otherwise foraging continues. Transition decisions use no randomness.

On entry, FoodLureEffect moves up to three eligible ground items from unoccupied, reachable tiles within square radius two onto the creature's tile in fixed source order. Items are moved, not copied; inventories and terrain remain unchanged. Staying in the state does not repeat this effect. No eligible food means no relocation.

Final Design Summary

Design 2 is selected throughout. SRP separates helper responsibilities; the configurable diet supports OCP; CreatureState and Consumable provide abstraction boundaries. Existing food effects are reused, including their probabilities. Deterministic targeting therefore does not imply deterministic food outcomes.

The verification suite covers 16 scenarios and 70 checks, including blocked food, stale pickup, timed effects and framework integration. This rationale covers member B; the other states' transitions and entry effects belong to the wider group design.